

Relatório Técnico da 3ª Interação: Gerenciador de uma Farmácia

Larsen Tivane¹, Jesusnilson Mateus¹, Lorena Araújo¹

¹ Instituto Superior de Engenharia do Porto, Rua Dr. António Bernardino de Almeida, 431
4249-015 Porto, Portugal

{1231948, 1241774, 1231658}@isep.ipp.pt

Abstract: Este projeto apresenta o desenvolvimento de um Sistema de Gestão de Farmácia destinado a apoiar as operações diárias através da informatização dos seus processos essenciais. Desenvolvido em C++ com arquitetura Model-View-Controller (MVC), o sistema permite gerir produtos, medicamentos, funcionários, vendas, stock, adicionar receitas, além de gerar relatórios. O resultado é uma aplicação funcional que responde às principais necessidades de uma farmácia, aplicando conceitos de engenharia de software, programação em C++ com suporte ao paradigma orientado a objetos e modelação UML.

Keywords: gestão de farmácia; C++; orientação a objetos; arquitetura MVC; UML; diagramas de sequência; registo de vendas; validação de receitas médicas; gestão de stock; persistência de dados.

1 Introdução

A gestão eficiente de uma farmácia requer o controlo rigoroso de produtos, medicamentos, vendas, receitas, e funcionários, garantindo que toda a informação se mantém organizada, atualizada e acessível. Quando estas tarefas são realizadas manualmente ou através de ferramentas pouco integradas, podem surgir erros operacionais, dificuldades no acompanhamento do stock e limitações na gestão das atividades diárias, comprometendo a qualidade do serviço prestado.

Com o objetivo de responder a estas necessidades, foi desenvolvido um Sistema de Gestão de Farmácia capaz de centralizar e automatizar as principais operações realizadas numa farmácia. O sistema permite gerir produtos e medicamentos, registar vendas, validar receitas médicas quando necessário, controlar automaticamente o stock, gerir funcionários e consultar relatórios de atividade, contribuindo para uma maior eficiência e fiabilidade dos processos.

O projeto foi desenvolvido em C++, recorrendo aos princípios da programação orientada a objetos e aplicando conceitos de análise de requisitos, modelação UML,

arquitetura de software e implementação de sistemas. Este relatório apresenta o trabalho realizado ao longo do projeto, descrevendo o problema identificado, a solução proposta, a arquitetura adotada, os diagramas desenvolvidos, as principais decisões de implementação e os resultados obtidos através da validação do sistema final.

2 Requisitos Funcionais e Não Funcionais

Os requisitos apresentados de seguida representam as funcionalidades e características que o sistema deve cumprir, tendo em conta o código implementado, a execução da aplicação e o contexto académico do projeto

2.1 Requisitos Funcionais

Os requisitos funcionais descrevem as ações que o sistema permite realizar e as principais regras de funcionamento da aplicação. O diagrama seguinte apresenta os principais requisitos funcionais do sistema, distinguindo as funcionalidades disponíveis para o funcionário e as permissões adicionais atribuídas ao gestor. Desta forma, é possível observar que o gestor herda as operações realizadas pelo funcionário, mas possui também acesso à gestão de stock, gestão de funcionários e geração de relatórios.

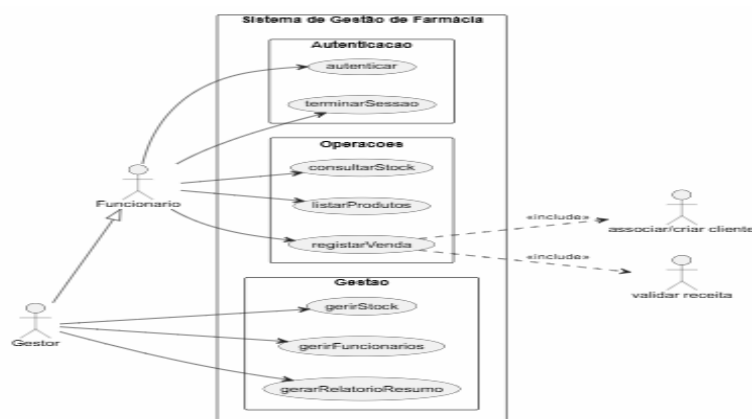


Figura 1. Diagrama dos Requisitos Funcionais

2.2 Requisitos Não Funcionais

Os requisitos não funcionais descrevem características de qualidade e restrições importantes para o funcionamento do sistema.

- O sistema deve ser simples de utilizar através de uma interface por consola.
- O sistema deve apresentar mensagens de erro compreensíveis.
- O sistema deve validar os dados antes de realizar operações importantes.
- O sistema deve restringir operações sensíveis apenas a gestores.
- O sistema deve estar organizado em entidades, repositório, controlador e vistas.
- O sistema deve funcionar localmente, sem necessidade de base de dados.

- O sistema deve ter desempenho adequado para pequenos volumes de dados.
- O sistema deve manter a integridade dos dados, evitando stock negativo, vendas inválidas e clientes inconsistentes.

3 Estrutura e Implementação do Sistema

Esta secção apresenta a estrutura e implementação do sistema desenvolvido para a gestão de uma farmácia. São descritos os requisitos funcionais e não funcionais, os modelos elaborados nas iterações anteriores (modelo de classes, casos de uso e diagramas de sequência), a arquitetura da aplicação, os principais componentes implementados e os testes realizados.

O sistema foi desenvolvido em C++, recorrendo aos princípios da programação orientada a objetos e a uma organização baseada no padrão Model-View-Controller (MVC). Esta abordagem permite separar a interface com o utilizador, a lógica de negócio e a gestão dos dados, tornando o sistema mais organizado e fácil de manter.

A interação com o utilizador é realizada através de menus na consola, permitindo executar operações como gestão de produtos, funcionários, vendas, receitas médicas, stock e relatórios. Cada funcionalidade segue um conjunto de passos simples e intuitivos, garantindo uma utilização eficiente do sistema.

Para apoiar a análise e implementação da solução, foram utilizados vários modelos apresentados na 2.^a iteração. Estes modelos permitiram representar a estrutura do sistema, as interações entre os utilizadores e as principais operações realizadas pela aplicação.

Ao longo desta secção serão ainda apresentados os componentes mais relevantes da implementação, alguns excertos de código representativos e os testes efetuados para validar o correto funcionamento das funcionalidades desenvolvidas.

3.1 Modelo de Domínio

Relativamente ao modelo de domínio apresentado na primeira iteração, foram efetuadas algumas alterações para o alinhar com a implementação final do sistema. As principais modificações incluem a introdução da entidade Farmacia como elemento central do domínio, a substituição de Gerente por Gestor, a revisão das relações de especialização entre Gestor e Funcionario e entre Medicamento e Produto, bem como a remoção da entidade Fornecedor, uma vez que esta funcionalidade não foi implementada na versão final da aplicação.

A relação entre Venda e ItemVenda foi também refinada para representar uma composição, evidenciando que os itens de venda apenas existem no contexto de uma venda. Por fim, a associação entre Venda e Receita foi ajustada para indicar que a utilização de receita médica é

opcional, ocorrendo apenas quando são vendidos medicamentos sujeitos a prescrição médica. Estas alterações permitiram obter um modelo mais consistente com as funcionalidades efetivamente desenvolvidas e com a estrutura final da aplicação.

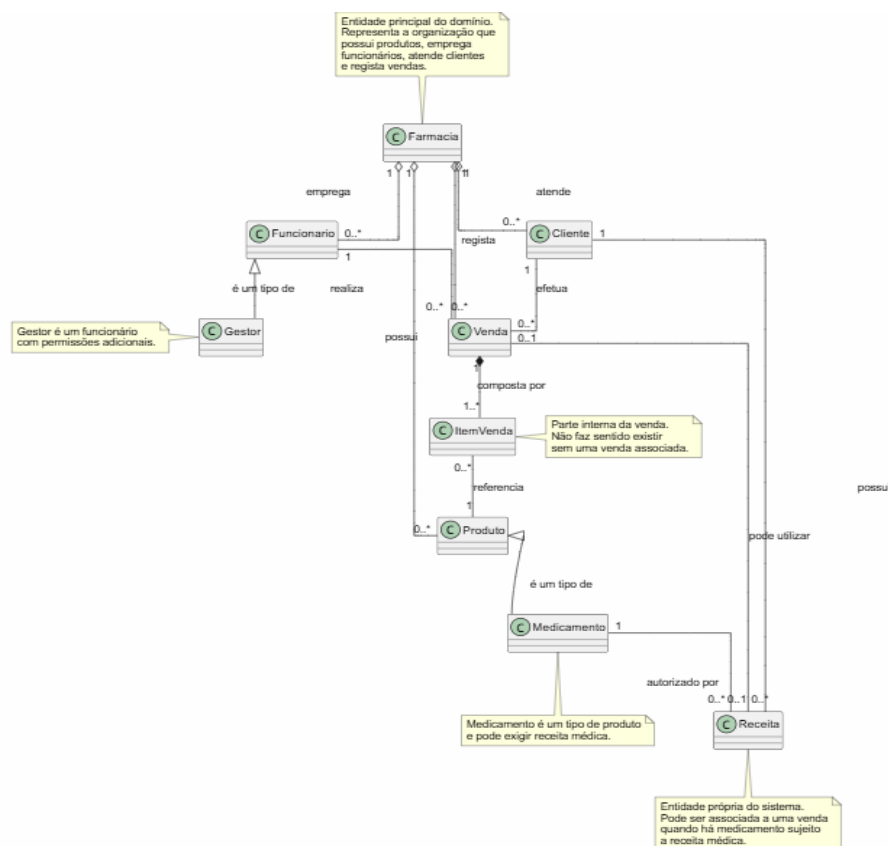


Figura 2. Modelo de Domínio

4 Casos de Uso

Após a identificação das principais entidades do domínio, torna-se importante representar as funcionalidades disponibilizadas pelo sistema e os diferentes tipos de utilizadores que interagem com a aplicação. Para esse efeito, foi elaborado um diagrama de casos de uso.

Os casos de uso comuns podem ser realizados por qualquer utilizador autenticado, enquanto os casos de uso de gestão estão reservados ao gestor, que possui permissões adicionais para gerir stock, funcionários e relatórios.

4.1 Caso de Uso: autenticar

Tabela 1 – Especificação do Caso de Uso "Autenticar Utilizador".

Caso de Uso	Autenticar Utilizador
Ator Principal	Funcionário ou Gestor
Objetivo	Permitir o acesso ao sistema.
Pré-condições	O funcionário ou gestor deve existir nos dados carregados pelo sistema.
Fluxo Principal	1. O utilizador introduz o username e a password. 2. O sistema verifica as credenciais introduzidas. 3. Se as credenciais forem válidas, o utilizador é autenticado. 4. O sistema disponibiliza as funcionalidades de acordo com o perfil do utilizador.
Fluxo Alternativo	1. O utilizador introduz credenciais inválidas. 2. O sistema rejeita a autenticação. 3. É apresentada uma mensagem de erro ao utilizador.
Pós-condições	Existe um utilizador autenticado no sistema e apto a utilizar as funcionalidades permitidas pelo seu perfil.

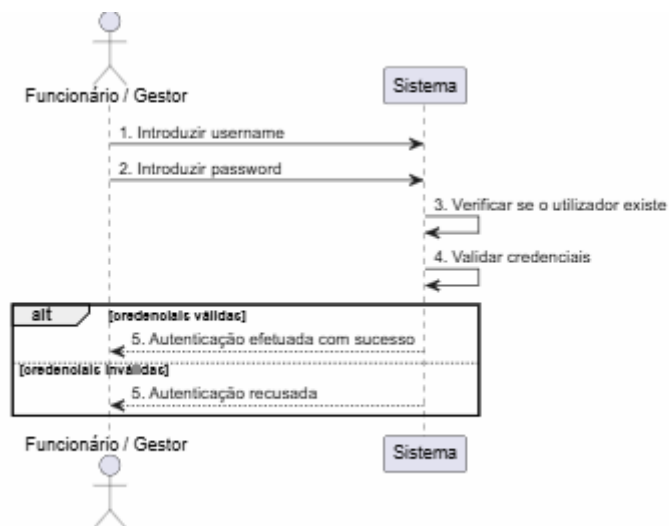


Figura 3. Diagrama do UC Autenticar

4.2 Caso de Uso: terminarSessao

Tabela 2 – Especificação do Caso de Uso "Terminar Sessão".

Caso de Uso	Terminar Sessão
Ator Principal	Funcionário ou Gestor
Objetivo	Terminar a sessão atual.
Pré-condições	Deve existir um utilizador autenticado no sistema.
Fluxo Principal	1. O utilizador seleciona a opção de terminar sessão. 2. O sistema remove a informação do utilizador autenticado. 3. O sistema encerra a sessão atual.
Pós-condições	O sistema fica sem qualquer utilizador autenticado e regressa ao estado inicial de autenticação.

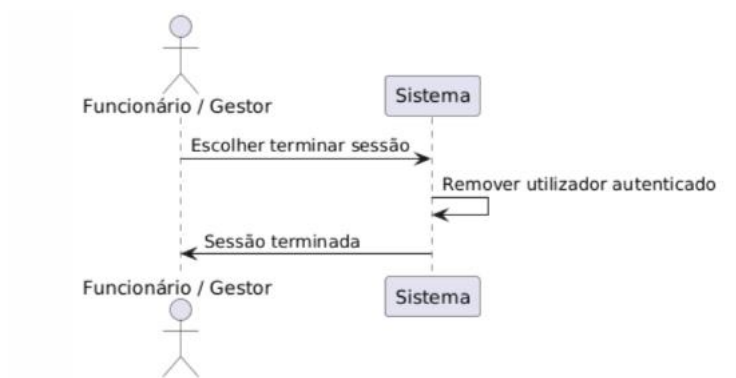


Figura 4. Caso de Uso: terminarSessao

4.3 Caso de Uso: listarProdutos

Tabela 3 – Especificação do Caso de Uso "Consultar Produtos".

Caso de Uso	Consultar Produtos
Ator Principal	Funcionário ou Gestor
Objetivo	Consultar os produtos disponíveis no sistema.
Pré-condições	O utilizador deve estar autenticado.
Fluxo Principal	1. O utilizador seleciona a opção de listar produtos. 2. O sistema consulta os produtos registados. 3. O sistema apresenta a lista de produtos disponíveis. 4. Para cada produto são apresentados dados como nome, categoria, preço e quantidade em stock.
Pós-condições	A lista de produtos registados é apresentada no terminal para consulta.

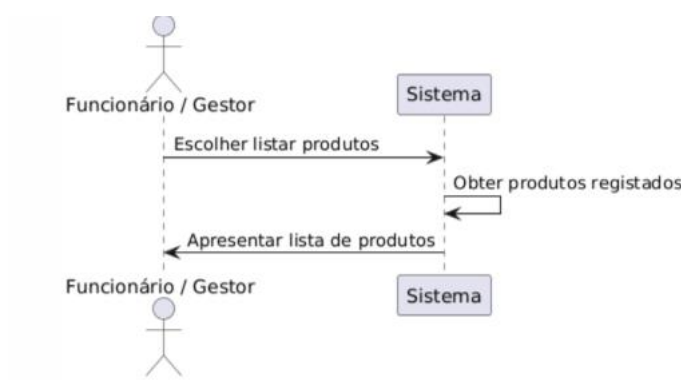


Figura 5. Caso de Uso: Listar produtos

4.4 Caso de Uso: consultarStock

Tabela 4 – Especificação do Caso de Uso "Consultar Stock de Produto".

Caso de Uso	Consultar Stock de Produto
Ator Principal	Funcionário ou Gestor
Objetivo	Consultar a quantidade em stock de um produto.
Pré-condições	O utilizador deve estar autenticado.
Fluxo Principal	1. O utilizador seleciona a opção de consultar stock. 2. O utilizador escolhe o produto pretendido. 3. O sistema verifica a existência do produto. 4. O sistema apresenta a quantidade disponível em stock.
Fluxo Alternativo	1. O utilizador seleciona um produto inexistente. 2. O sistema apresenta uma mensagem de erro indicando que o produto não foi encontrado.
Pós-condições	A quantidade em stock do produto é apresentada no terminal.

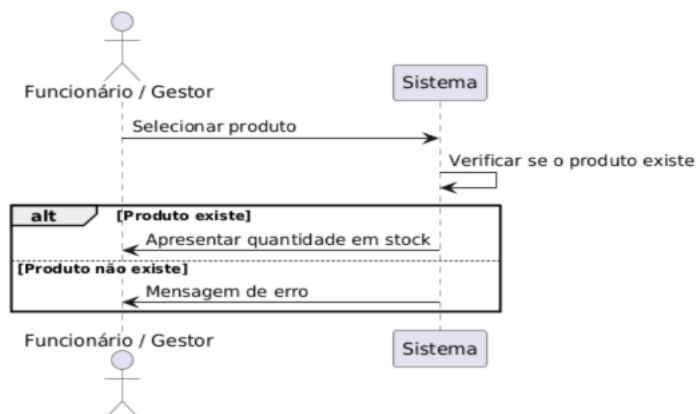


Figura 6: Caso de Uso: consultarStock

4.5 Caso de Uso: registrarVenda

Tabela 5 – Especificação do Caso de Uso "Registrar Venda".

Caso de Uso	Registrar Venda
Ator Principal	Funcionário ou Gestor
Objetivo	Registrar uma venda no sistema.
Pré-condições	O utilizador deve estar autenticado.
Fluxo Principal	1. O utilizador seleciona a opção de registrar venda. 2. O sistema apresenta os produtos disponíveis. 3. O utilizador seleciona os produtos e as respetivas quantidades. 4. O sistema verifica se os produtos existem e se existe stock suficiente. 5. O sistema solicita os dados do cliente associado à venda. 6. Caso o cliente não exista, o sistema cria um novo registo de cliente. 7. Se algum medicamento exigir receita médica, o sistema solicita e valida a receita. 8. O sistema calcula o valor total da venda. 9. O sistema regista a venda. 10. O sistema atualiza o stock dos produtos vendidos.
Fluxo Alternativo	1. Se não existir stock suficiente para algum produto, a venda é cancelada. 2. Se um medicamento exigir receita médica e esta não for validada, a venda é recusada.

	3. Se o utilizador não seleccionar qualquer produto, a venda não é registada.
Pós-condições	A venda fica registada no sistema, o stock é atualizado e, quando aplicável, fica associada uma receita ao cliente.

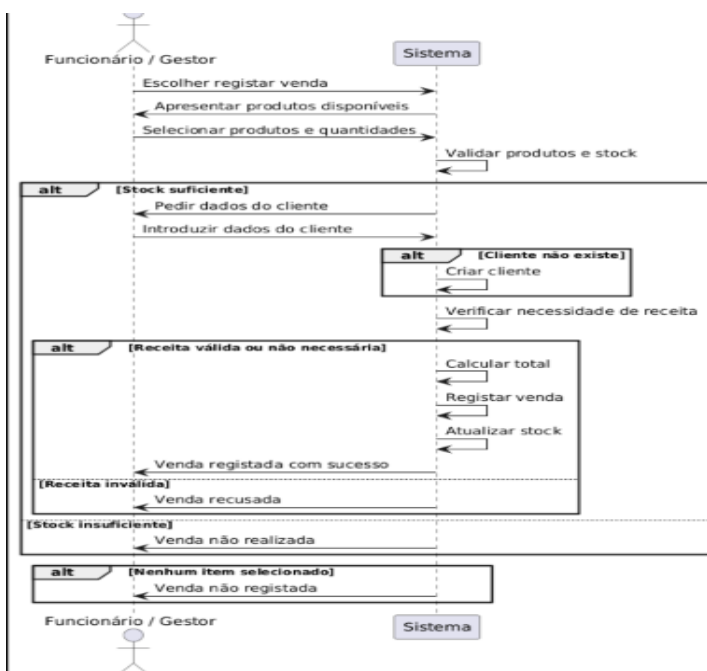


Figura 7: Caso de Uso: registrarVenda

4.6 Caso de Uso: gerirStock

Tabela 5 – Especificação do Caso de Uso "Gerir Stock e Produtos".

Caso de Uso	Gerir Stock e Produtos
Ator Principal	Gestor
Objetivo	Gerir os produtos e as respetivas quantidades em stock.
Pré-condições	O utilizador deve estar autenticado como gestor.
Fluxo Principal	1. O gestor acede à funcionalidade de gestão de stock. 2. O sistema apresenta as opções disponíveis para gestão de produtos. 3. O gestor pode adicionar novos produtos, indicar se um produto é ou não um

	medicamento e alterar as quantidades existentes em stock. 4. O gestor introduz os dados necessários para a operação pretendida. 5. O sistema valida os dados introduzidos. 6. O sistema atualiza a informação dos produtos e do stock.
Fluxo Alternativo	1. Se os dados introduzidos forem inválidos, o sistema apresenta uma mensagem de erro. 2. Se o produto indicado não existir, o sistema informa o gestor da situação. 3. Se a operação resultar numa quantidade de stock inválida, o sistema rejeita a alteração e apresenta uma mensagem de erro.
Pós-condições	O catálogo de produtos e as respetivas quantidades em stock ficam atualizados.

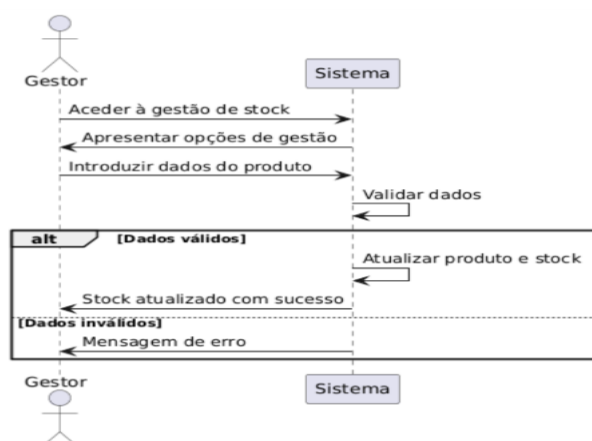


Figura 8: Caso de Uso: gerirStock

4.7 Caso de Uso: gerirFuncionarios

Tabela 6 – Especificação do Caso de Uso "Gerir Utilizadores".

Caso de Uso	Gerir Utilizadores
Ator Principal	Gestor
Objetivo	Gerir os utilizadores do sistema.
Pré-condições	O utilizador deve estar autenticado como gestor.
Fluxo Principal	1. O gestor seleciona a opção de gestão de funcionários.

	2. O sistema apresenta as opções disponíveis para gestão de utilizadores. 3. O gestor pode adicionar funcionários, adicionar gestores ou remover funcionários existentes. 4. Para adicionar um utilizador, o gestor introduz os dados necessários. 5. Para remover um utilizador, o gestor seleciona o funcionário pretendido. 6. O sistema valida a operação e atualiza a lista de utilizadores.
Fluxo Alternativo	1. Se o gestor tentar remover a própria conta autenticada, o sistema impede a operação e apresenta uma mensagem de erro. 2. Se os dados introduzidos forem inválidos, o sistema rejeita a operação e informa o utilizador.
Pós-condições	A lista de funcionários e gestores registados no sistema é atualizada.

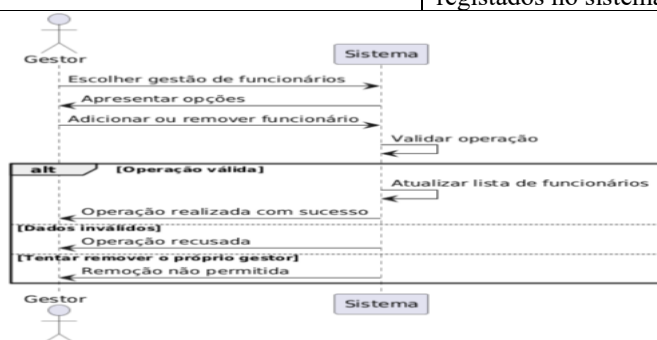


Figura 9: Caso de Uso: gerir Funcionarios

4.8 Caso de Uso: gerarRelatorioResumo

Tabela 7 – Especificação do Caso de Uso "Consultar Relatório Resumo".

Caso de Uso	Consultar Relatório Resumo
Ator Principal	Gestor
Objetivo	Consultar um resumo geral da atividade da farmácia.
Pré-condições	O utilizador deve estar autenticado como gestor.

Fluxo Principal	1. O gestor seleciona a opção de relatório. 2. O sistema recolhe os dados necessários para o resumo da atividade. 3. O sistema calcula o número total de produtos registados. 4. O sistema calcula o número total de vendas efetuadas. 5. O sistema calcula o número total de itens vendidos. 6. O sistema calcula o valor total faturado. 7. O sistema apresenta o relatório resumo ao gestor.
Fluxo Alternativo	1. Se o utilizador não possuir permissões de gestor, o sistema impede o acesso à funcionalidade e apresenta uma mensagem de erro.
Pós-condições	O relatório resumo da atividade da farmácia é apresentado no terminal.

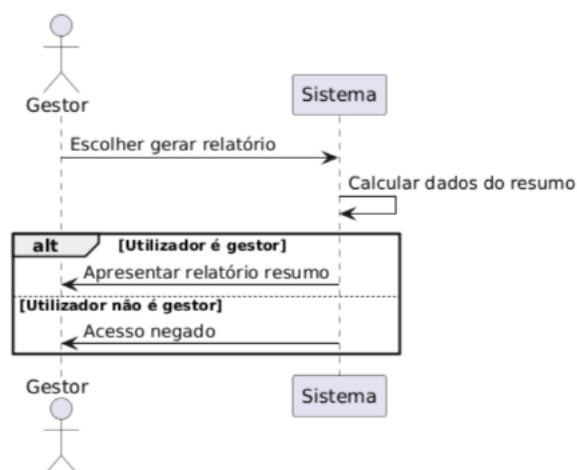


Figura 10: Caso de Uso: gerarRelatorioResumo

5 Diagrama de Classes

Após a definição do modelo de domínio, foi desenvolvido o diagrama de classes para representar a estrutura implementada na aplicação conforme demonstrado na Figura 10. Este diagrama descreve as classes, os seus atributos, métodos e relações.

Face às versões anteriores, foram introduzidas alterações para refletir a implementação final do sistema, destacando-se a classe Farmacia como elemento central de gestão dos dados. Foram também refinadas as relações entre Venda, Receita, ItemVenda, Produto e Medicamento, de forma a representar corretamente as regras de negócio e funcionalidades implementadas, como autenticação, gestão de stock, processamento de vendas e validação de receitas médicas.

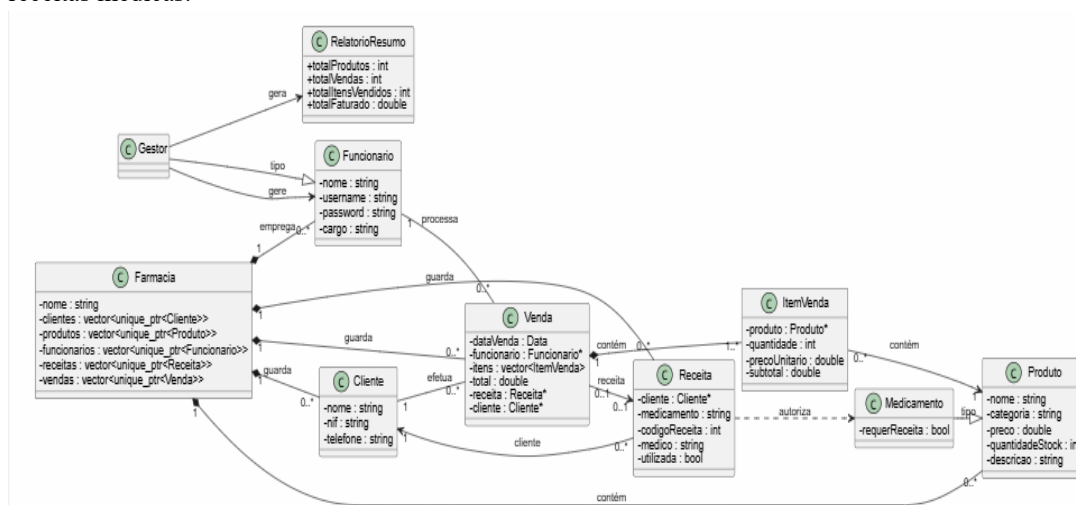


Figura 10- Diagrama de Classes

O diagrama de classes evidencia a organização interna da aplicação e as relações existentes entre as diferentes classes. A classe Farmacia assume o papel central da aplicação, sendo responsável pelo armazenamento das coleções principais do sistema, incluindo produtos, clientes, funcionários, receitas e vendas.

A classe Funcionario representa os utilizadores responsáveis pelo registo de vendas, enquanto Gestor surge como uma especialização de Funcionario, dispondo de permissões adicionais para operações administrativas. Da mesma forma, Medicamento é uma especialização de Produto, acrescentando a indicação sobre a obrigatoriedade de receita médica.

A classe Venda representa uma transação realizada na farmácia, estando associada a um cliente, a um funcionário e, opcionalmente, a uma receita médica. Cada venda contém um ou mais objetos ItemVenda, responsáveis por armazenar a informação relativa aos produtos vendidos, quantidades e valores associados.

Por sua vez, a classe Receita permite representar a validação de medicamentos sujeitos a prescrição médica, armazenando informações relativas ao cliente, medicamento, médico prescritor e estado de utilização da receita.

6 Diagrama do Sistema

O diagrama seguinte apresenta a organização do ambiente de execução do sistema, evidenciando os principais elementos envolvidos no seu funcionamento.

É possível observar a interação entre o utilizador, a aplicação e o mecanismo de armazenamento de dados, permitindo compreender de forma simplificada como o sistema é executado e onde a informação é persistida.

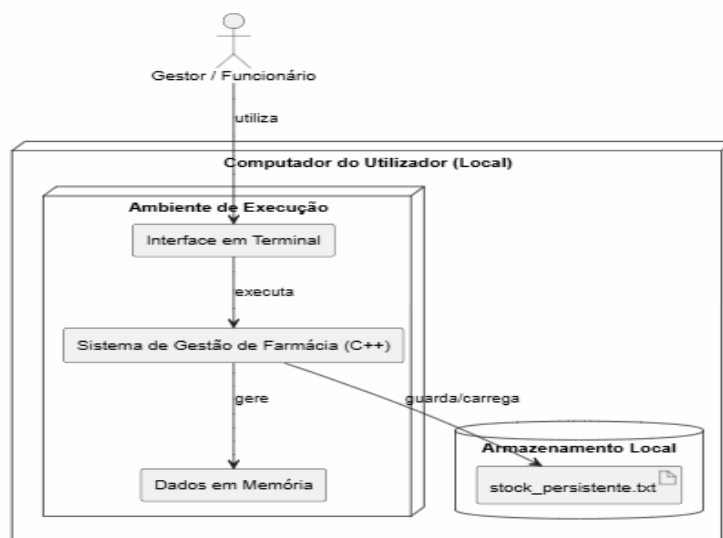


Figura 11- Diagrama de Sistema

Foi desconsiderada a persistência de todos os dados em ficheiros .txt, uma vez que, na implementação atual, o sistema mantém a maior parte da informação em memória durante a execução. Apenas o stock é guardado e carregado através do ficheiro stock_persistente.txt, por ser o dado que necessita de continuidade entre execuções da aplicação. Esta decisão simplifica a implementação e é adequada ao contexto académico do projeto.

7 Diagrama de Componentes

O diagrama de componentes representa a organização da aplicação segundo uma arquitetura MVC. A camada View é responsável pela interação com o utilizador, o Controller coordena as operações e aplica as regras de negócio, enquanto o Model contém as entidades principais do sistema. O repositório faz a ligação entre o Controller e os dados do modelo, garantindo também a persistência local do stock.

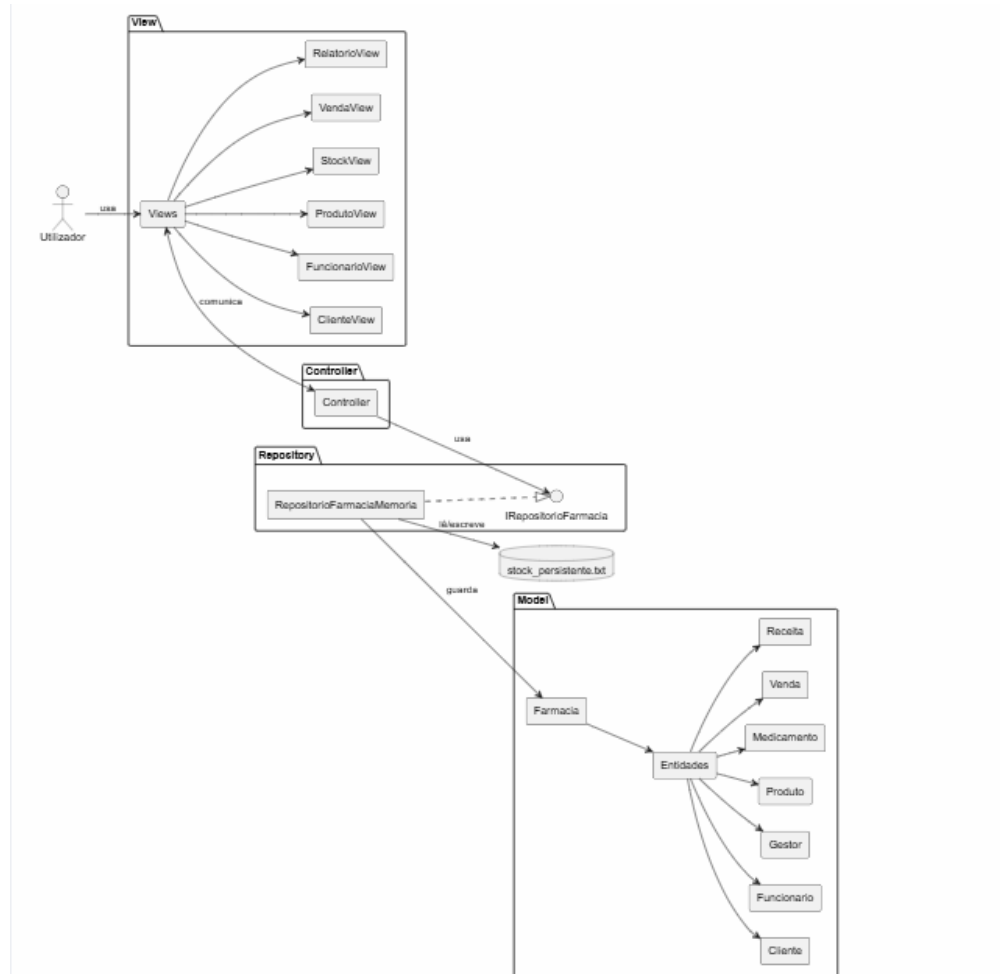


Figura 12. Diagrama de componentes

Como se observa no diagrama, o sistema encontra-se organizado segundo uma arquitetura MVC. A camada View concentra as classes responsáveis pela interação com o utilizador através da consola. A camada Controller coordena as operações solicitadas pelas views e comunica com o repositório através da interface **IRepositorioFarmacia**. O componente **RepositorioFarmaciaMemoria** implementa essa interface, guarda os dados em memória

através da classe Farmacia e realiza a leitura e escrita do ficheiro stock_persistente.txt, utilizado para persistir o stock entre execuções. Desta forma, a aplicação mantém uma separação clara entre interface, controlo, modelo e armazenamento de dados.

8 Diagrama de interfaces

O diagrama de interfaces apresenta a estrutura de navegação da aplicação e as principais funcionalidades disponibilizadas aos utilizadores após a autenticação. Através deste diagrama é possível observar a organização do menu principal e as operações que podem ser executadas no sistema, incluindo consulta de produtos, registo de vendas, consulta de clientes, gestão de stock, gestão de funcionários e geração de relatórios.

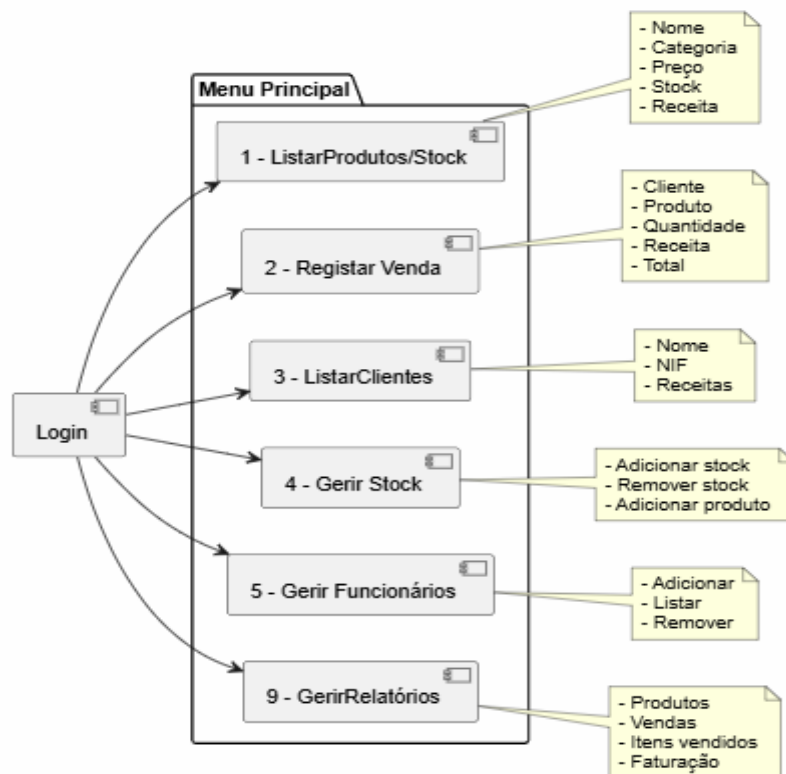


Fig. 13. Diagrama de interface do sistema

Após a autenticação, o utilizador pode aceder ao menu principal da aplicação e navegar pelas diferentes funcionalidades disponíveis. Estas incluem a consulta de produtos e stock, o registo de vendas, a consulta de clientes, a gestão de stock e funcionários, bem como a geração

de relatórios. Cada funcionalidade disponibiliza as operações necessárias para apoiar as atividades de gestão da farmácia e o controle das suas operações diárias.

9 Diagramas de Sequência

Os diagramas de sequência apresentados têm como objetivo representar, de forma temporal e detalhada, o fluxo de interação entre os diferentes componentes do sistema de gestão de farmácia durante a execução de funcionalidades essenciais. Estes diagramas permitem compreender como os atores e os módulos do sistema comunicam entre si ao longo de cada processo.

No conjunto seguinte, será apresentado primeiro o diagrama de sequência relativo ao funcionário, onde se demonstra o processo de registo de uma venda, incluindo a verificação de stock e a atualização automática da informação. De seguida, será apresentado o diagrama de sequência do gestor, evidenciando as operações administrativas, como a gestão de stock, produtos e relatórios, reforçando a separação de responsabilidades entre perfis de utilizador.

9.1 Diagrama de Sequência- Registrar de Venda(Funcionário/Gestor)

O diagrama seguinte representa o processo de registo de uma venda realizado pelo funcionário no sistema de gestão de farmácia. O fluxo distingue a venda de produtos comuns e medicamentos sujeitos a receita médica. No caso de medicamentos controlados, é realizada a validação da receita antes da confirmação da venda. Após o registo, o sistema atualiza automaticamente o stock.

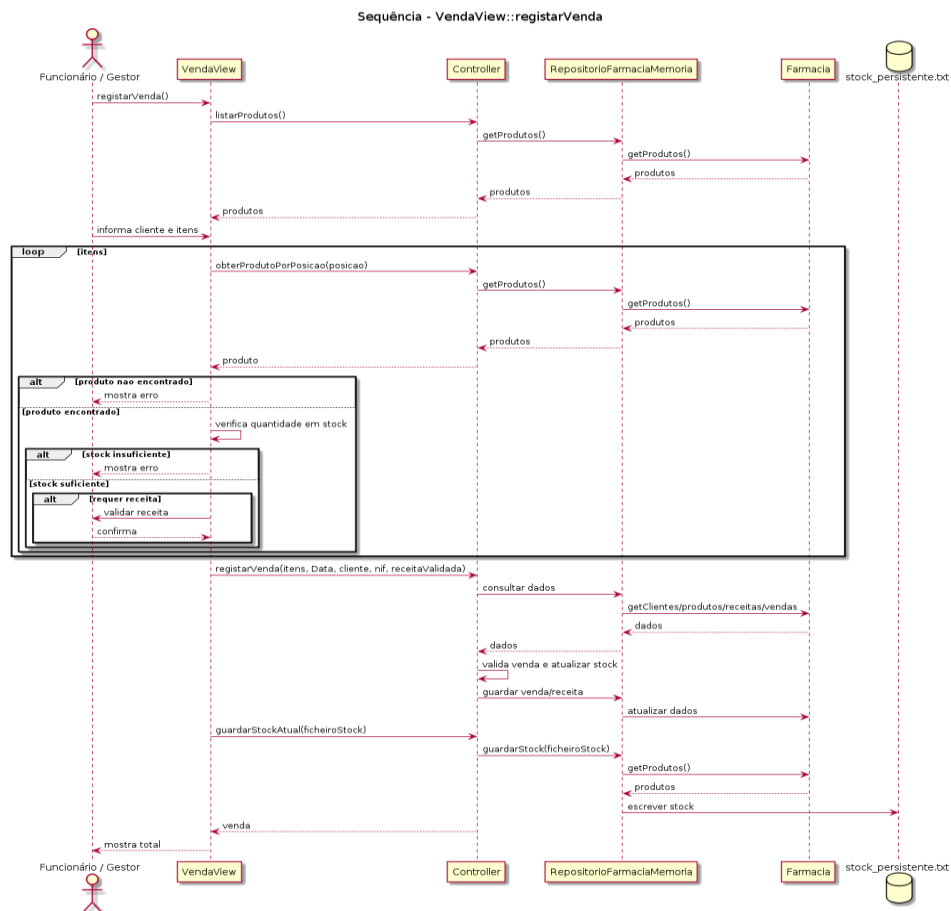


Fig.14 Diagrama de Sequência- RegistrarVenda

9.2 Diagrama de Sequência- ListarStock (Funcionário)

A consulta de stock permite aos utilizadores autenticados verificar a quantidade disponível de um determinado produto registado na farmácia. Para isso, o sistema apresenta inicialmente a lista de produtos existentes, permitindo ao utilizador seleccionar o produto pretendido. Após a selecção, o sistema valida a autenticação do utilizador e procura o produto nos dados armazenados. Caso o produto seja encontrado, é apresentada a respetiva quantidade em stock; caso contrário, é apresentada uma mensagem de erro.

A Figura 15 apresenta o diagrama de sequência correspondente à funcionalidade de consulta de stock, ilustrando as interações entre o utilizador, a interface, o controlador e os componentes responsáveis pelo armazenamento dos dados.

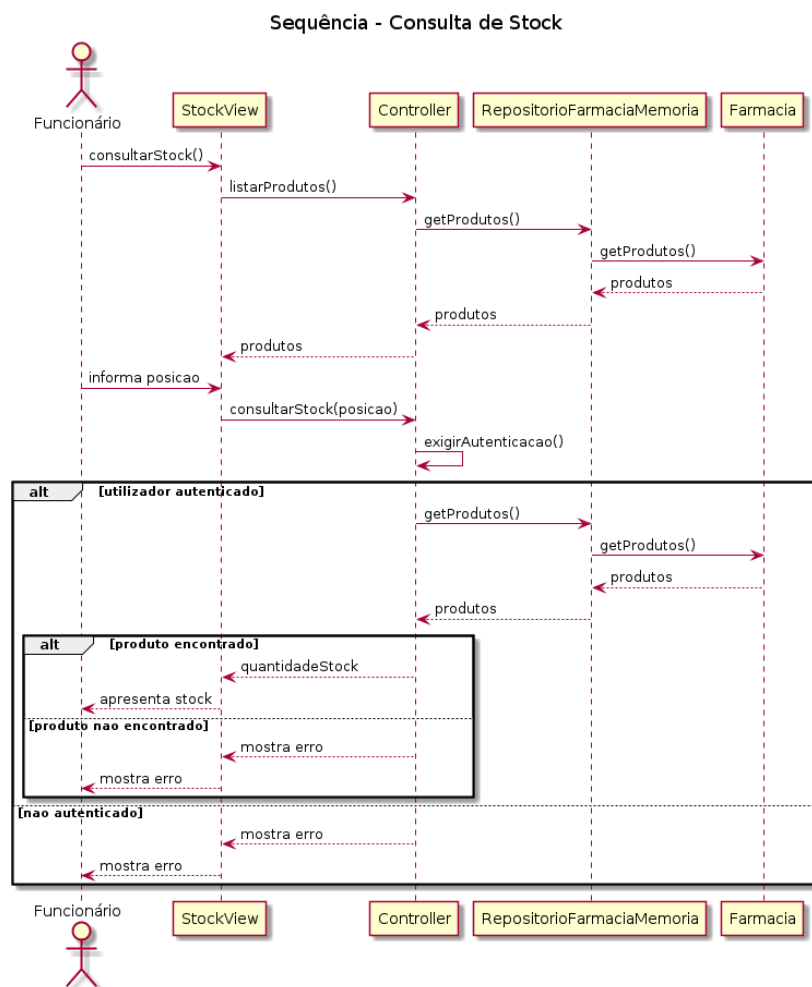


Figura 15 – Diagrama de sequência da funcionalidade de consulta de stock.

9.3 Diagrama de Sequência- Gerir de Stock (Gestor)

A funcionalidade de gerir stock está disponível apenas para gestores e permite adicionar ou remover stock, bem como registar novos produtos.

O sistema valida as permissões do utilizador e os dados introduzidos antes de efetuar alterações. Após a operação, a informação é atualizada e guardada no ficheiro de persistência de stock.

A Figura 16 apresenta o diagrama de sequência desta funcionalidade.

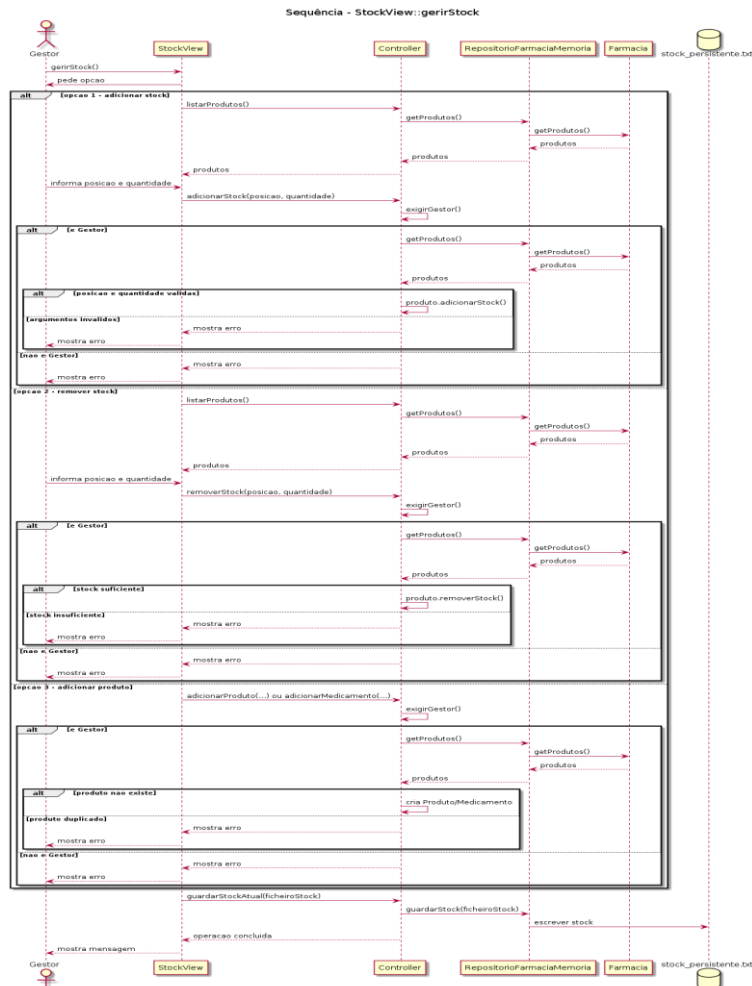


Fig.16 Diagrama de Sequência- Gerir Stock (Gestor)

O fluxo inicia-se quando o gestor selecciona uma operação de gestão de stock. O sistema valida as permissões do utilizador e os dados introduzidos, verificando também a existência de stock suficiente ou de produtos duplicados, conforme a operação escolhida.

Após a execução, as alterações são guardadas no ficheiro de stock e é apresentada uma mensagem de confirmação. Em caso de erro, o sistema informa o utilizador da situação ocorrida.

9.4 Diagrama de Sequência- Gerir Relatórios (Gestor)

A funcionalidade de gerir relatórios permite aos utilizadores com perfil de gestor consultar informação resumida sobre a atividade da farmácia. Através desta funcionalidade é possível obter indicadores estatísticos calculados a partir dos produtos e vendas registados no sistema.

Para gerar o relatório, o sistema valida as permissões do utilizador, recolhe os dados armazenados e calcula automaticamente os valores necessários para a apresentação do resumo. A Figura 17 apresenta o diagrama de sequência associado à geração do relatório resumo.

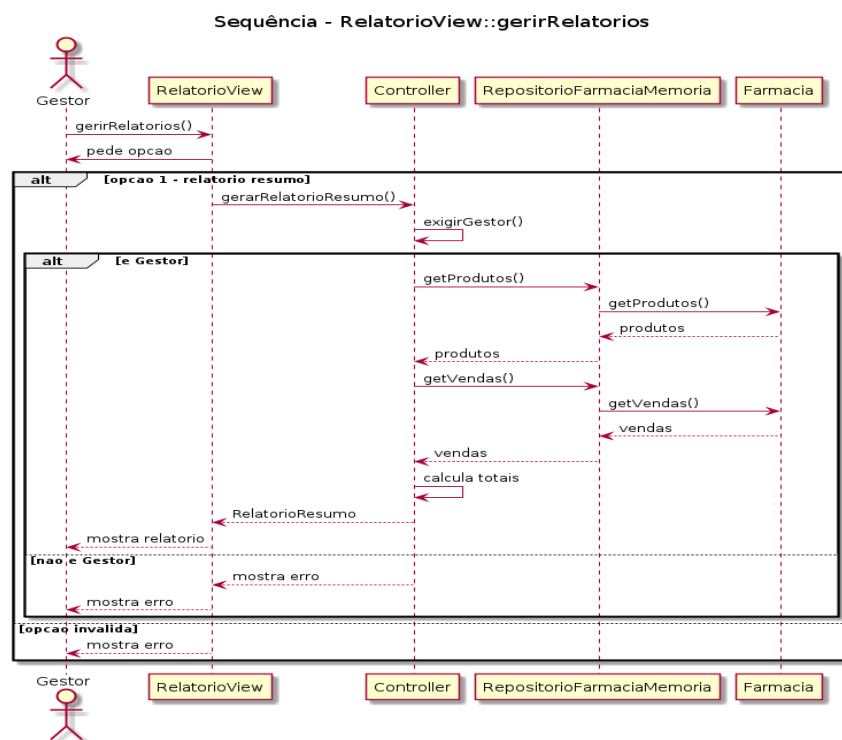


Fig.17. Diagrama de Sequência- Gerir Relatórios

O gestor solicita a geração do relatório resumo e o sistema valida as suas permissões. Em seguida, recolhe os dados de produtos e vendas, calcula os principais indicadores (total de produtos, vendas, itens vendidos e valor faturado) e apresenta o relatório ao utilizador. Caso não existam permissões adequadas ou ocorra uma operação inválida, é apresentada uma mensagem de erro.

9.5 Diagrama de Sequência- GerirFuncionários (Gestor)

A funcionalidade de gerir funcionários permite aos gestores adicionar, consultar e remover utilizadores do sistema.

Antes de executar qualquer operação, o sistema verifica as permissões do utilizador e realiza as validações necessárias, como a disponibilidade do nome de utilizador ou a validade da seleção para remoção.

A Figura 18 apresenta o diagrama de sequência desta funcionalidade.

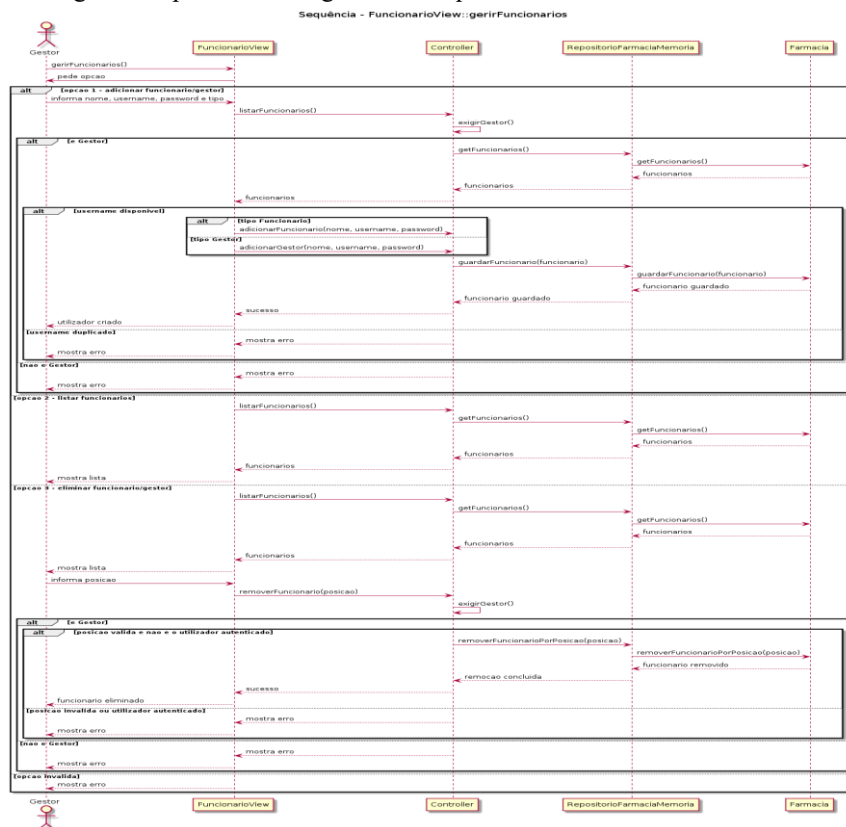


Fig.18 Diagrama de Sequência- Gerir Funcionários (Gestor)

O processo inicia-se quando o gestor acede à funcionalidade e seleciona a operação pretendida. O sistema valida as permissões necessárias e executa a ação escolhida, permitindo criar, consultar ou remover utilizadores.

Durante a execução, são realizadas validações específicas, como a verificação da disponibilidade do nome de utilizador ou da posição selecionada para remoção. No final, o

sistema apresenta uma mensagem de sucesso ou, em caso de erro, informa o utilizador sobre a situação ocorrida.

10 Testes

10.1 Integração do GoogleTest

Para garantir a qualidade e a fiabilidade da aplicação desenvolvida, foi utilizado o framework GoogleTest para a implementação dos testes unitários. A integração foi realizada através do CMake, recorrendo ao módulo FetchContent para descarregar automaticamente a biblioteca quando esta não se encontra instalada no sistema.

O ficheiro de configuração responsável pela integração do GoogleTest encontra-se apresentado na Listagem 1.

```

include(FetchContent)

find_package(GTest CONFIG QUIET)

if(NOT GTest_FOUND)
    FetchContent_Declare(
        googletest
        URL https://github.com/google/googletest/archive/refs/tags/v1.14.0.zip
    )
    set(gtest_force_shared_crt ON CACHE BOOL "" FORCE)
    FetchContent_MakeAvailable(googletest)
endif()

add_executable(ProjectTester
    main_test.cpp
    ProdutoTest.cpp
    ClienteTest.cpp
    FuncionarioTest.cpp
    ControllerTest.cpp
    VendaTest.cpp
    RelatorioTest.cpp
)

target_link_libraries(ProjectTester PRIVATE PharmacyCore
GTest::gtest)

add_test(NAME ProjectTester COMMAND ProjectTester)

```

Listagem 1. Integração do GoogleTest

Após a obtenção da biblioteca, foi criado o executável de testes ProjectTester, responsável por compilar e executar todos os testes desenvolvidos para as diferentes funcionalidades do sistema.

10.2 Testes Unitários

Os testes unitários foram organizados em diferentes ficheiros, de acordo com as entidades e funcionalidades principais do sistema, como demonstra a figura.

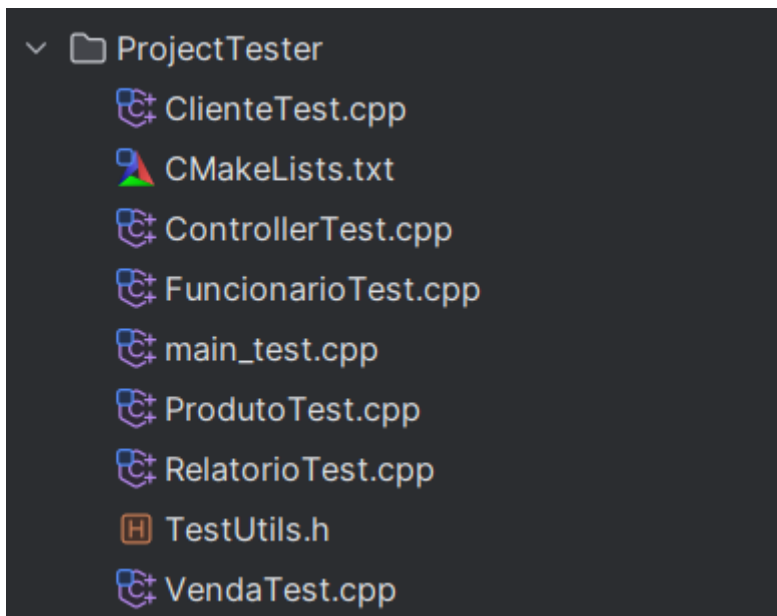


Figura 19. Ficheiros de Teste

O ficheiro main_test.cpp constitui o ponto de entrada da framework GoogleTest.

```
#include <gtest/gtest.h>

int main(int argc, char** argv) {
    testing::InitGoogleTest(&argc, argv);
    return RUN_ALL_TESTS();
}
```

Este ficheiro é responsável por inicializar a framework e executar automaticamente todos os testes definidos na aplicação.

10.2.1 Testes da Classe Produto

Os testes associados à classe Produto verificam a criação de produtos, a validação dos dados introduzidos e as operações de gestão de stock.

```
TEST(ProdutoTest, ValidaDadosEStock) {
    Produto produto("Termometro", "Equipamento", 12.50, 10);

    EXPECT_EQ("Termometro", produto.getNome());
    EXPECT_TRUE(produto.verificarDisponibilidade(10));
    EXPECT_FALSE(produto.verificarDisponibilidade(11));

    produto.adicionarStock(5);
    EXPECT_EQ(15, produto.getQuantidadeStock());
}
```

```

    produto.removerStock(4);
    EXPECT_EQ(11, produto.getQuantidadeStock());

    EXPECT_THROW(Produto invalido("AB", "Equipamento", 12.50, 10),
std::invalid_argument);
    EXPECT_THROW(produto.removerStock(12), std::invalid_argument);
}

```

10.2.2 Testes da Classe Cliente

Os testes da classe Cliente validam a criação de clientes e a verificação dos dados identificativos.

```

TEST(ClienteTest, ValidaNifEIgualdade) {
    Cliente cliente("Ana Paciente", "123456789", "912345678");
    Cliente mesmoNif("Outro Nome", "123456789");
    Cliente mesmoNome("Ana Paciente");

    EXPECT_EQ("123456789", cliente.getNif());
    EXPECT_TRUE(cliente == mesmoNif);
    EXPECT_TRUE(cliente == mesmoNome);

    EXPECT_THROW(Cliente invalido("An", "123456789"),
std::invalid_argument);
    EXPECT_THROW(Cliente invalido("Ana Paciente", "123"),
std::invalid_argument);
}

```

10.2.3 Testes da Classe Funcionario

Os testes da classe Funcionario garantem o correto funcionamento do mecanismo de autenticação.

```

TEST(FuncionarioTest, Autenticacao) {
    Funcionario funcionario("Joao Silva", "joao", "abcd",
"Funcionario");

    EXPECT_TRUE(funcionario.autenticar("joao", "abcd"));
    EXPECT_FALSE(funcionario.autenticar("joao", "errada"));
    EXPECT_TRUE(funcionario == "joao");

    EXPECT_THROW(Funcionario invalido("Jo", "joao", "abcd",
"Funcionario"), std::invalid_argument);
    EXPECT_THROW(Funcionario invalido("Joao Silva", "js", "abcd",

```

```
"Funcionario"),                                std::invalid_argument);
}
```

10.2.4 Testes do Controller

Os testes do Controller verificam as regras de negócio implementadas, incluindo permissões de acesso e operações de gestão.

```
TEST(ControllerTest,          ControlaPermissoesDeStock)    {
    AmbienteController          ambiente;
    Gestor&  gestor  =  ambiente.controller.adicionarGestor("Maria
Gestora",          "gestora",          "1234");
    Funcionario&          funcionario          =
ambiente.controller.adicionarFuncionario("Joao  Silva",  "joao",
"abcd");
    ambiente.controller.adicionarProduto("Termometro",
"Equipamento",          12.50,          10);

    EXPECT_EQ(&funcionario,  ambiente.controller.autenticar("joao",
"abcd"));
    EXPECT_EQ(10,          ambiente.controller.consultarStock(1));
    EXPECT_THROW(ambiente.controller.adicionarStock(1,          3),
std::runtime_error);

    EXPECT_EQ(&gestor,          ambiente.controller.autenticar("gestora",
"1234"));
    ambiente.controller.adicionarStock(1,          5);
    EXPECT_EQ(15,          ambiente.controller.consultarStock(1));
}

TEST(ControllerTest,          RemoveFuncionarioComRegrasDeGestor)    {
    AmbienteController          ambiente;
    ambiente.controller.adicionarGestor("Maria  Gestora",  "gestora",
"1234");
    ambiente.controller.adicionarFuncionario("Joao  Silva",  "joao",
"abcd");
    ambiente.controller.adicionarFuncionario("Carlos  Temporario",
"carlos",          "temp");

    ambiente.controller.autenticar("gestora",          "1234");
    EXPECT_EQ(3,  ambiente.controller.listarFuncionarios().size());
}
```

```

    ambiente.controller.removeFuncionario(3);
    EXPECT_EQ(2, ambiente.controller.listarFuncionarios().size());
    EXPECT_EQ(nullptr, ambiente.controller.autenticar("carlos",
"temp"));

    ambiente.controller.autenticar("gestora", "1234");
    EXPECT_THROW(ambiente.controller.removeFuncionario(1),
std::runtime_error);
}

```

10.2.5 Testes de Registo de Vendas

Os testes relacionados com as vendas validam o cálculo dos valores faturados, a atualização automática do stock e a associação de receitas médicas quando necessário.

```

TEST(VendaTest, CriaClienteReceitaEAtualizaStock) {
    AmbienteController ambiente;
    ambiente.controller.adicionarGestor("Maria Gestora", "gestora",
"1234");
    Funcionario& funcionario =
ambiente.controller.adicionarFuncionario("Joao Silva", "joao",
"abcd");
    ambiente.controller.adicionarProduto("Termometro",
"Equipamento", 12.50, 10);
    Medicamento& medicamento =
ambiente.controller.adicionarMedicamento(
    "Antibiotico", "Medicamento", 8.0, 5, true);

    ambiente.controller.autenticar("joao", "abcd");
    Venda& venda = ambiente.controller.registarVenda(
        {{1, 2}, {2, 1}},
        Data(19, 5, 2026),
        "Ana", "Paciente",
        true);

    EXPECT_EQ(&funcionario, venda.getFuncionario());
    EXPECT_DOUBLE_EQ(33.0, venda.getTotal());
    ASSERT_NE(nullptr, venda.getClient());
    EXPECT_EQ("Ana Paciente", venda.getNomePaciente());
}

```

```

    ASSERT_NE(nullptr, venda.getReceita());
    EXPECT_EQ(venda.getClient(), venda.getReceita()->getClient());
    EXPECT_EQ(medicamento.getNome(), venda.getReceita()-
>getMedicamento());
    EXPECT_TRUE(venda.getReceita()->foiUtilizada());
    EXPECT_EQ(8, ambiente.controller.consultarStock(1));
    EXPECT_EQ(4, ambiente.controller.consultarStock(2));
    EXPECT_EQ(1, ambiente.controller.listarClientes().size());
    EXPECT_EQ(1, ambiente.controller.listarReceitas().size());
}

TEST(VendaTest, RecusaMedicamentoSemReceitaValidada) {
    AmbienteController ambiente;
    ambiente.controller.adicionarFuncionario("Joao Silva", "joao",
"abcd");
    ambiente.controller.adicionarMedicamento("Antibiotico",
"Medicamento", 8.0, 5, true);

    ambiente.controller.autenticar("joao", "abcd");
    EXPECT_THROW(
        ambiente.controller.registarVenda({{1, 1}}, Data(19, 5,
2026), "Ana Paciente", false),
        std::runtime_error);
    EXPECT_TRUE(ambiente.controller.listarVendas().empty());
    EXPECT_EQ(5, ambiente.controller.consultarStock(1));
}

```

10.2.6 Testes de Relatórios

Os testes dos relatórios garantem que os indicadores estatísticos apresentados ao gestor são calculados corretamente.

```

TEST(RelatorioTest, ResumeVendas) {
    AmbienteController ambiente;
    ambiente.controller.adicionarGestor("Maria Gestora", "gestora",
"1234");
    ambiente.controller.adicionarFuncionario("Joao Silva", "joao",
"abcd");
    ambiente.controller.adicionarProduto("Termometro",

```

```

"Equipamento",          12.50,          10);
    ambiente.controller.adicionarProduto("Seringa", "Equipamento",
1.25,          20);

    ambiente.controller.autenticar("joao", "abcd");
    ambiente.controller.registarVenda({{1, 2}, {2, 4}}, Data(19, 5,
2026), "Ana", "Paciente");

    ambiente.controller.autenticar("gestora", "1234");
    RelatorioResumo          resumo          =
ambiente.controller.gerarRelatorioResumo();

    EXPECT_EQ(2,          resumo.totalProdutos);
    EXPECT_EQ(1,          resumo.totalVendas);
    EXPECT_EQ(6,          resumo.totalItensVendidos);
    EXPECT_DOUBLE_EQ(30.0, resumo.totalFaturado);
}

```

10.2.7 Resultado da Execução dos Testes

Após a implementação dos testes unitários, foi realizada a sua execução através da framework GoogleTest. A execução permitiu validar as principais funcionalidades e regras de negócio da aplicação, incluindo autenticação, gestão de stock, gestão de funcionários, registo de vendas, validação de receitas médicas e geração de relatórios.

A Listagem a baixo apresenta o resultado obtido durante a execução dos testes.

```

[=====] Running 8 tests from 6 test suites.
[-----] Global test environment set-up.

[-----] 1 test from RelatorioTest
[ RUN ] RelatorioTest.ResumeVendas
[ OK ] RelatorioTest.ResumeVendas (1 ms)

[-----] 2 tests from VendaTest
[ RUN ] VendaTest.CriaClienteReceitaEAtualizaStock
[ OK ] VendaTest.CriaClienteReceitaEAtualizaStock (0 ms)
[ RUN ] VendaTest.RecusaMedicamentoSemReceitaValidada
[ OK ] VendaTest.RecusaMedicamentoSemReceitaValidada (2 ms)

```



```

[-----]      2      tests      from      ControllerTest
[  RUN              ]      ControllerTest.ControlaPermissoesDeStock
[          OK  ]      ControllerTest.ControlaPermissoesDeStock (0 ms)
[  RUN              ]      ControllerTest.RemoveFuncionarioComRegrasDeGestor
[          OK  ]      ControllerTest.RemoveFuncionarioComRegrasDeGestor (0 ms)

[-----]      1      test      from      FuncionarioTest
[  RUN              ]      FuncionarioTest.Autenticacao
[          OK  ]      FuncionarioTest.Autenticacao (0 ms)

[-----]      1      test      from      ClienteTest
[  RUN              ]      ClienteTest.ValidaNifEIgualdade
[          OK  ]      ClienteTest.ValidaNifEIgualdade (0 ms)

[-----]      1      test      from      ProdutoTest
[  RUN              ]      ProdutoTest.ValidaDadosEStock
[          OK  ]      ProdutoTest.ValidaDadosEStock (0 ms)

[=====] 8 tests from 6 test suites ran. (1113 ms total)
[  PASSED  ] 8 tests.

```

Os resultados demonstram que todos os testes foram executados com sucesso, não tendo sido detetadas falhas nas funcionalidades avaliadas. Foram validadas as operações de autenticação, gestão de produtos e stock, gestão de funcionários, registo de vendas, validação de receitas médicas e geração de relatórios.

A execução dos testes permitiu confirmar que as regras de negócio implementadas se comportam conforme esperado e que os principais componentes da aplicação apresentam um funcionamento consistente e fiável.

11 Implementação das Funcionalidades

Após a definição da arquitetura, dos diagramas UML e da validação do sistema através de testes unitários, procede-se à apresentação das principais funcionalidades implementadas na aplicação. Esta secção tem como objetivo demonstrar o funcionamento prático do sistema, ilustrando as operações disponibilizadas aos utilizadores através de exemplos de execução e capturas de ecrã.

São apresentadas as funcionalidades mais relevantes da aplicação, incluindo autenticação, gerir produtos e stock, registo de vendas, gerir funcionários e relatórios. Para cada funcionalidade é descrito o seu objetivo, o fluxo de utilização e os principais resultados obtidos durante a execução do programa.

11.1 Autenticação

A autenticação constitui o mecanismo de controlo de acesso da aplicação. Apenas utilizadores previamente registados podem aceder às funcionalidades do sistema. Dependendo do perfil autenticado, são disponibilizadas diferentes permissões e operações. Segue abaixo demonstração da execução:

```
Sistema de Gestao de Farmacia
Nome da farmacia:ling

Credenciais de teste: gestora/1234 ou joao/abcd
Username:gestora
Password:1234
Sessao iniciada como Maria Gestora (Gestor).

=== Sistema de Gestao de Farmacia - ling ===
1 - Listar produtos/stock
2 - Registrar venda
3 - Listar clientes
4 - Gerir stock
5 - Gerir funcionarios
9 - Gerir relatorios
10 - Terminar sessao
0 - Sair
Opcao:|
```

Figura 20 – Autenticação bem-sucedida de um utilizador com perfil de gestor e apresentação do menu principal da aplicação.

A autenticação é executada em três passos principais. Em primeiro lugar, o utilizador introduz o nome de utilizador. De seguida, introduz a palavra-passe associada à conta. Por fim, o sistema valida as credenciais e, caso sejam válidas, cria uma sessão autenticada e apresenta o menu correspondente ao perfil do utilizador.

Este mecanismo garante que apenas utilizadores autorizados podem aceder às funcionalidades da aplicação, permitindo ainda distinguir as permissões disponíveis para funcionários e gestores.

11.2 Consulta de Produtos e Stock

A funcionalidade de consulta de produtos e stock permite aos utilizadores visualizar todos os produtos registados na farmácia, bem como as respetivas informações relevantes para a gestão diária da atividade.

Ao selecionar a opção correspondente no menu principal, o sistema apresenta uma tabela contendo o nome do produto, a categoria, o preço unitário, a quantidade disponível em stock e a indicação sobre a necessidade de receita médica.

A Figura 21 apresenta um exemplo da listagem de produtos existente no sistema.

```
=== Sistema de Gestao de Farmacia - ling ===
1 - Listar produtos/stock
2 - Registrar venda
3 - Listar clientes
4 - Gerir stock
5 - Gerir funcionarios
9 - Gerir relatorios
10 - Terminar sessao
0 - Sair
Opcao:1
```

Produtos em stock					
Opcao	Nome	Categoria	Preco	Stock	Receita
1	Termometro	Equipamento	12.50	10	Nao
2	Mascara Cirurgica	Protecao	0.50	198	Nao
3	Paracetamol	Medicamento	3.75	40	Nao
4	Antibiotico	Medicamento	8.00	5	Sim

Figura 21 – Listagem dos produtos registados na farmácia, incluindo categoria, preço, quantidade em stock e necessidade de receita médica.

A informação apresentada permite aos utilizadores consultar rapidamente o estado atual do inventário da farmácia. No exemplo apresentado é possível observar produtos de diferentes categorias, incluindo equipamentos de saúde, materiais de proteção e medicamentos.

Adicionalmente, o sistema identifica os medicamentos sujeitos a receita médica, permitindo distinguir os produtos que exigem validação adicional durante o processo de venda. Esta funcionalidade contribui para uma gestão mais eficiente do stock e para o cumprimento das regras associadas à dispensa de medicamentos.

11.3 Registo de Vendas

O registo de vendas constitui uma das funcionalidades centrais da aplicação. Esta funcionalidade permite efetuar vendas de produtos da farmácia, associando cada transação a um cliente e validando automaticamente as regras de negócio necessárias.

Durante o processo de venda, o sistema apresenta os produtos disponíveis, solicita os dados do cliente e permite selecionar os produtos e respetivas quantidades. Sempre que um medicamento sujeito a receita médica é selecionado, o sistema verifica a existência de uma receita válida antes de concluir a operação.

A Figura 22 apresenta um exemplo de uma venda envolvendo um medicamento sujeito a receita médica.

```
Opcao:2

Produtos em stock
Opcao  Nome                Categoria    Preco    Stock  Receita
1      Termometro           Equipamento 12.50    10     Nao
2      Mascara Cirurgica     Protecao     0.50    198    Nao
3      Paracetamol          Medicamento 3.75     40     Nao
4      Antibiotico          Medicamento 8.00     5      Sim

Nome do cliente (v para voltar):Linda
NIF do cliente (v para voltar):154236889

Opcao do produto (0 para terminar venda) (v para voltar):4
Quantidade (v para voltar):2
Este medicamento requer receita.
A receita e valida? (sim/nao):sim

Opcao do produto (0 para terminar venda) (v para voltar):0
Venda registada com sucesso em nome de Linda. Receita registad
a para Antibiotico. Total: 16.00 EUR
```

Figura 22 – Registo de uma venda com validação de receita médica para um medicamento sujeito a prescrição.

No exemplo apresentado, foi efetuada uma venda para a cliente Linda, identificada através do respetivo NIF. Durante a operação foi selecionado o medicamento "Antibiotico", configurado no sistema como sujeito a receita médica.

Após a introdução da quantidade pretendida, o sistema solicitou a validação da receita. Uma vez confirmada a sua validade, a venda foi registada com sucesso, tendo sido criada a receita associada e calculado automaticamente o valor total da transação, correspondente a 16,00 euros.

Esta funcionalidade assegura não apenas o registo das vendas efetuadas, mas também o cumprimento das restrições associadas à dispensa de medicamentos sujeitos a prescrição médica, contribuindo para a fiabilidade e segurança do sistema.

10.4 Listar Clientes

A funcionalidade Listar Clientes permite visualizar os clientes atualmente registados no sistema. Esta informação é particularmente útil durante o processo de venda, uma vez que permite identificar clientes já existentes e verificar a existência de receitas médicas associadas.

Ao seleccionar a opção correspondente no menu principal, o sistema apresenta uma listagem organizada dos clientes registados, incluindo o nome, o número de identificação fiscal (NIF) e a informação relativa às receitas médicas associadas.

A Figura 23 apresenta um exemplo da consulta de clientes existente na aplicação.

```
=== Sistema de Gestao de Farmacia - ling ===
1 - Listar produtos/stock
2 - Registrar venda
3 - Listar clientes
4 - Gerir stock
5 - Gerir funcionarios
9 - Gerir relatorios
10 - Terminar sessao
0 - Sair
Opcao:3

Clientes registados
Opcao  Nome                NIF                Receitas
1      Ana Paciente        123456789          Sem receitas
```

Figura 23 – Consulta dos clientes registados no sistema, apresentando o nome, NIF e informação relativa às receitas médicas.

Esta funcionalidade facilita a gestão das vendas e das receitas médicas, uma vez que permite identificar rapidamente clientes já registados e consultar a informação associada ao seu histórico dentro do sistema.

11.5 Gerir Stock

A gestão de stock é uma funcionalidade exclusiva dos utilizadores com perfil de gestor. Esta

funcionalidade permite adicionar novos produtos ao sistema, identificar medicamentos sujeitos a receita médica e atualizar as quantidades existentes em stock.

A Figura 24 apresenta um exemplo da adição de um novo produto ao sistema. Durante este processo, o gestor introduz as informações necessárias, incluindo nome, categoria, preço, quantidade em stock e indicação sobre a necessidade de receita médica.

Embora a funcionalidade também permita alterar quantidades existentes em stock, ou eliminar, a operação apresentada é representativa das principais validações e mecanismos implementados nesta área da aplicação.

```
Opcao:4
  1 - Adicionar stock
  2 - Remover stock
  3 - Adicionar produto
Opcao (v para voltar):3
Nome (v para voltar):Omeprazol
Preco (v para voltar):8.55
Stock inicial (v para voltar):5
  1 - Produto comum
  2 - Medicamento
Tipo (v para voltar):2
Requer receita? (1 sim, 0 nao) (v para voltar):sim
Valor invalido. Tente novamente.
Requer receita? (1 sim, 0 nao) (v para voltar):1
Produto adicionado com sucesso.
```

Figura 24 – Adição de um novo medicamento através da funcionalidade de gestão de stock.

No exemplo apresentado foi adicionado o medicamento "Omeprazol", com um preço unitário de 8,55 euros e uma quantidade inicial de cinco unidades em stock. Durante o processo foi também indicada a necessidade de receita médica para a sua venda.

Esta funcionalidade desempenha um papel fundamental na gestão do inventário da farmácia, permitindo manter atualizada a informação relativa aos produtos disponíveis e às respetivas quantidades em stock.

10.6 Gerir Funcionários

A funcionalidade de gestão de funcionários encontra-se disponível exclusivamente para utilizadores com perfil de gestor. Esta funcionalidade permite adicionar novos funcionários ou gestores ao sistema, bem como consultar e remover utilizadores existentes.

A Figura 25 apresenta um exemplo da criação de um novo funcionário. Durante este processo, o gestor introduz o nome, o nome de utilizador, a palavra-passe e o tipo de conta a criar. Após a validação dos dados, o novo utilizador fica imediatamente disponível para autenticação no sistema.

Os utilizadores adicionados podem iniciar sessão utilizando as credenciais definidas durante o processo de criação, desde que a aplicação permaneça em execução. Atualmente, a persistência de funcionários não se encontra implementada, pelo que os utilizadores criados são armazenados apenas em memória e deixam de existir quando a aplicação é encerrada.

```
Opcao:5
  1 - Adicionar funcionario/gestor
  2 - Listar funcionarios
  3 - Eliminar funcionario/gestor
Opcao (v para voltar):1
  Nome (v para voltar):Linda
  Username (v para voltar):Wendy
  Password (v para voltar):bvcx
  1 - Funcionario
  2 - Gestor
Tipo (v para voltar):1
  Utilizador criado com sucesso.
```

Figura 25 – Criação de um novo funcionário através da funcionalidade de gestão de funcionários.

No exemplo apresentado foi criado um novo utilizador com o nome "Linda", nome de utilizador "Wendy" e perfil de funcionário. Após a conclusão da operação, o sistema confirma a criação da conta e esta passa a poder ser utilizada para autenticação durante a execução atual da aplicação.

Esta funcionalidade permite ao gestor controlar os acessos ao sistema e diferenciar as permissões atribuídas a funcionários e gestores.

10.7 Gerir Relatórios

A funcionalidade de geração de relatórios encontra-se disponível apenas para utilizadores com perfil de gestor e permite obter uma visão global da atividade da farmácia. O relatório resumo agrega automaticamente informação relevante sobre os produtos registados, as vendas efetuadas, os itens vendidos e o valor total faturado.

Ao selecionar a opção correspondente no menu principal, o sistema calcula os indicadores estatísticos disponíveis e apresenta os resultados de forma organizada no terminal. Esta funcionalidade permite ao gestor acompanhar o estado geral da aplicação e obter rapidamente informação relevante para apoio à gestão da farmácia.

A Figura 26 apresenta um exemplo da geração de um relatório resumo.

```
=== Sistema de Gestao de Farmacia - ling ===
1 - Listar produtos/stock
2 - Registrar venda
3 - Listar clientes
4 - Gerir stock
5 - Gerir funcionarios
9 - Gerir relatorios
10 - Terminar sessao
0 - Sair
Opcao:9
  1 - Relatorio resumo
Opcao (v para voltar):1

Relatorio resumo
Produtos registados: 5
Vendas registadas: 0
Itens vendidos: 0
Total faturado: 0.00 EUR
```

Figura 26 – Geração do relatório resumo com os principais indicadores da atividade da farmácia.

No exemplo apresentado, o sistema indica que existem cinco produtos registados e que ainda não foram efetuadas vendas durante a execução atual da aplicação. Consequentemente, o número de vendas registadas, o total de itens vendidos e o valor faturado apresentam o valor zero.

O relatório é gerado automaticamente a partir dos dados armazenados pela aplicação, permitindo ao gestor consultar rapidamente informação consolidada sobre o funcionamento do sistema. Esta funcionalidade contribui para o acompanhamento da atividade da farmácia e para a análise dos resultados obtidos.

12 Discussão Final

O projeto resultou num sistema funcional para gestão de uma farmácia, incluindo autenticação de utilizadores, fazer a gestão de produtos e stock, registo de vendas, validação de receitas médicas, gestão de funcionários e criação de relatórios.

Durante o desenvolvimento foram efetuadas algumas alterações ao modelo inicial. A entidade Fornecedor foi removida por não ter sido implementada, enquanto a entidade Farmacia foi introduzida como elemento central do sistema. Também foram ajustadas as relações entre classes, destacando-se as especializações de Gestor em relação a Funcionario e de Medicamento em relação a Produto.

A arquitetura inspirada no padrão MVC contribuiu para uma melhor organização da aplicação e separação de responsabilidades. Além disso, os testes unitários realizados com GoogleTest permitiram validar as principais funcionalidades e regras de negócio implementadas.

Como limitação, apenas os dados de stock são atualmente persistidos em ficheiro, permanecendo outras informações apenas em memória durante a execução da aplicação.

De forma geral, a solução desenvolvida cumpre os objetivos definidos e demonstra a aplicação prática dos conceitos de análise, modelação, implementação e teste de software.

13 Conclusão

Este projeto teve como objetivo desenvolver uma aplicação para gerir uma farmácia, aplicando conceitos de análise UML, tendo como suporte o paradigma orientado a objetos, arquitetura de software e testes unitários.

A aplicação inclui funcionalidades como autenticação de utilizadores, gestão de produtos e stock, registo de vendas, validação de receitas médicas, gestão de funcionários e geração de relatórios. Os diagramas foram ajustados ao longo do desenvolvimento para refletir a implementação final, garantindo coerência entre a análise e a solução desenvolvida.

Os resultados demonstram que os objetivos definidos foram alcançados, tendo sido criada uma aplicação funcional e adequada às operações essenciais de uma farmácia. Como trabalho futuro, poderão ser adicionados mecanismos de persistência mais completos, novos relatórios e melhorias na interface de utilizador.

Em suma, o projeto permitiu consolidar conhecimentos de engenharia de software e desenvolver uma solução alinhada com os requisitos estabelecidos.